Day 9: AWS Database Services

RDS, DynamoDB, Redshift & Aurora

Today we unlock the power of AWS's purpose-built database ecosystem. Whether you're tracking orders, managing user sessions, or analyzing years of sales data — there's a right database for every job. Let's build the skills that cloud employers need.

ITNW 1009 — CLOUD ESSENTIALS

Today's Roadmap

What We'll Cover Today

By the end of this lesson, you'll be able to identify the right AWS database service for any real-world scenario — a skill employers look for in every cloud technician role.

Relational vs. NoSQL

Understand the fundamental structural difference between SQL and NoSQL data models.

Amazon RDS & Aurora

Explore managed relational engines and AWS's cloud-native high-performance database.

Amazon DynamoDB

Discover serverless NoSQL for millisecond-speed key-value workloads at any scale.

Amazon Redshift

See how petabyte-scale data warehousing powers business intelligence and analytics.

Why Databases Matter

Unlocking AWS Database Services

Databases Are Everywhere

Every time you order shoes online, check your bank balance, or scroll social media — a database is fetching your data behind the scenes. They store, structure, and retrieve the core information that enterprise applications depend on.

AWS Thinks Differently

Traditional IT forced every application onto one massive, expensive relational server. AWS takes a **purpose-built approach** — matching the right database engine to each specific workload. The result? Better performance, lower cost, and higher reliability.

- Relational — structured, relationship-driven data
- NoSQL — flexible, high-speed key-value data
- Analytical — petabyte-scale business intelligence

Key Vocabulary

Terms You Need to Know

Relational

Data organized into structured tables with rows, columns, and enforced relationships.

NoSQL

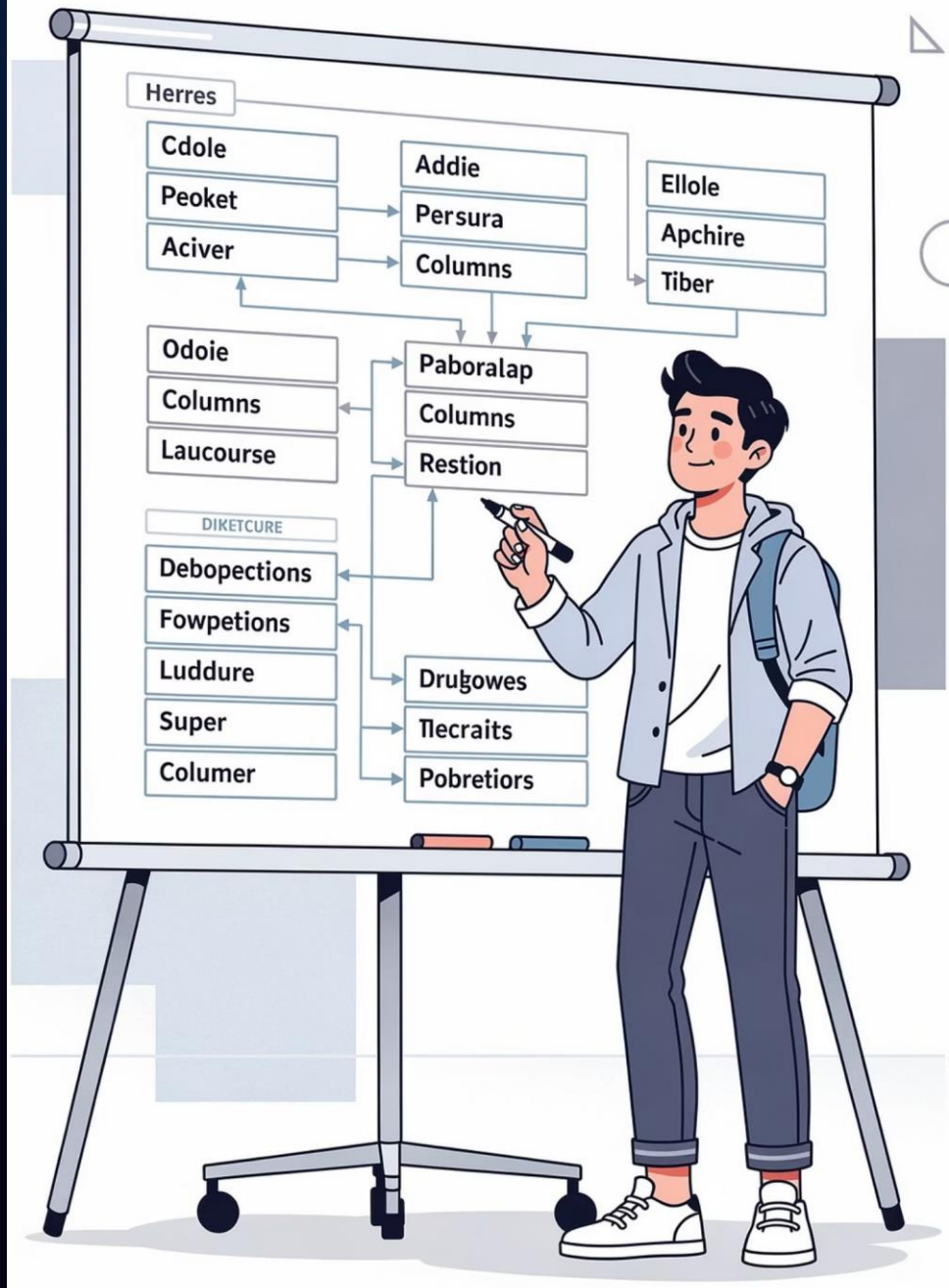
Flexible schema databases using key-value, document, or graph structures for speed.

Provisioned Throughput

Pre-reserving a fixed number of read/write capacity units per second in DynamoDB.

Data Warehouse

A centralized repository for large-scale analytical queries across historical data.



Chapter 1

Relational vs. NoSQL

Understanding the structural difference between these two data models is the foundation of every database decision you'll make in your cloud career.

Structural Comparison: Relational vs. NoSQL

🗂️ Relational (SQL)

Rigid table schemas with rows, columns, and enforced primary/foreign key relationships. Every record must match the defined structure exactly.

- Uses Structured Query Language (SQL)
- Enforces ACID compliance for data integrity
- Best for: accounting, HR management, inventory systems
- Think: standardized spreadsheet where every column is required

📦 NoSQL (Non-Relational)

Flexible schemas using key-value pairs or JSON documents. Each record can have a completely different structure — built for ultra-low latency.

- No enforced schema across every record
- Designed for horizontal scale and millisecond response
- Best for: gaming leaderboards, shopping carts, recommendation engines
- Think: a filing cabinet where each folder holds different contents

Real-World Analogy

The Library vs. The Parcel Locker

Relational = City Library

Books are cataloged with rigid entries — Author, Title, ISBN. Every record follows a strict schema. Finding all books by a specific author in 1995 requires following structural indexing rules. Accuracy is mandatory.

NoSQL = Parcel Locker Hub

Each locker holds a different package for a unique access code. One locker contains a small book; another holds three pairs of shoes. You enter your code — your **Partition Key** — and the locker opens instantly. Speed over structure.

Quick Check — Think It Through

Which Database Would You Choose?

> 💬 "If you are building a **bank transaction ledger** where every record must strictly match and link to a customer ID — would you choose Relational or NoSQL?"

Answer: Relational

Banking and financial records demand **ACID compliance** — Atomicity, Consistency, Isolation, and Durability. Every transaction must precisely link to a verified customer ID. A single error could mean a missing payment or a regulatory violation. Rigid schemas aren't a limitation here — they're a critical safety feature.

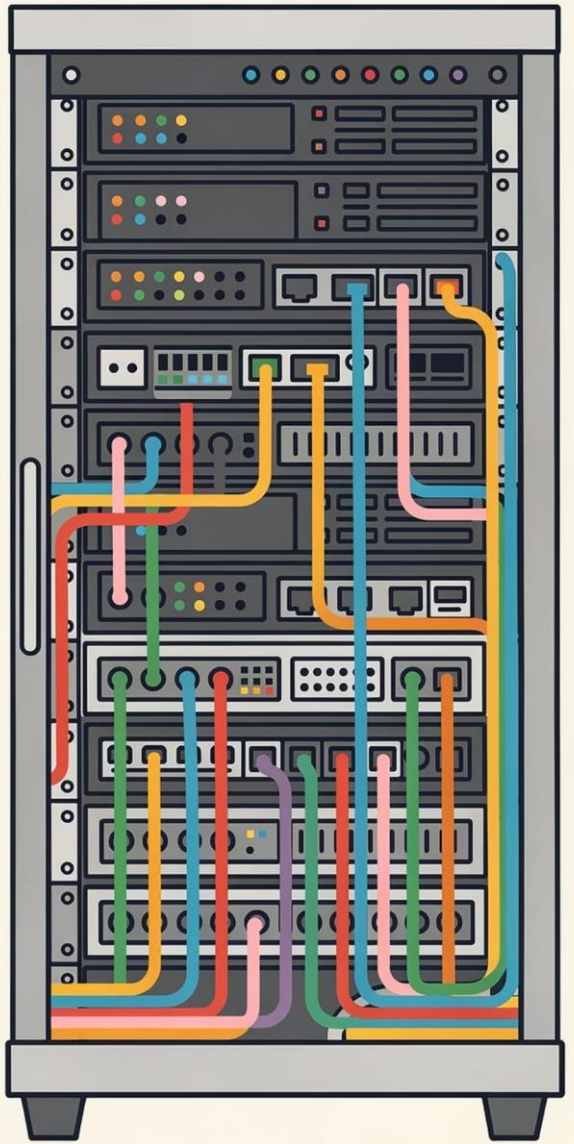
When Would You Pick NoSQL?

When speed trumps strict relationships. A real-time gaming leaderboard doesn't need a foreign key relationship to a customer billing table — it needs to return your score in under 5 milliseconds for millions of simultaneous players. That's the NoSQL sweet spot.

Chapter 2

Amazon RDS

The managed relational database service that handles the heavy lifting — so your team focuses on building applications, not maintaining servers.



Amazon Relational Database Service (RDS)

Running an on-premises database means buying hardware, installing the OS, applying patches, configuring backups, and responding to 2 AM failures. RDS automates all of that — you choose your engine and AWS handles the rest.

Fully Managed

AWS provisions infrastructure, applies OS patches, and manages database software updates automatically.

6 Supported Engines

MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora — pick the engine your team already knows.

Multi-AZ Standby

AWS creates a synchronous standby replica in a second Availability Zone. Failover is automatic — no manual intervention needed.

Automated Backups

Point-in-time recovery for up to 35 days, protecting your data without manual snapshot scheduling.

RDS Deep Dive

Multi-AZ: High Availability by Default

How Multi-AZ Works

When you enable **Multi-AZ deployment**, RDS automatically provisions and maintains a synchronous standby replica in a different Availability Zone. All writes to the primary database are simultaneously replicated to the standby.

If the primary instance fails — due to hardware failure, network disruption, or even a software crash — RDS automatically promotes the standby to primary and redirects your application's traffic. No manual failover required. Typical failover time: 60–120 seconds.

What RDS Manages For You

- Operating system patching and updates
- Database software version upgrades
- Automated daily backups with 35-day retention
- Synchronous replication across Availability Zones
- Hardware provisioning and replacement
- Monitoring and metric dashboards

RDS Analogy

The Restaurant Ordering System

Think of RDS like a **formal sit-down restaurant's ordering system**. Tables, waiters, customer orders, and bill totals are all strictly linked together in a multi-part order ticket. If a customer changes an item, the waiter updates the main ticket — and the kitchen, bar, and cashier all stay synchronized. Every relationship is tracked, and accuracy is mandatory.

Chapter 3

Amazon Aurora

AWS's cloud-native relational engine — delivering MySQL and PostgreSQL compatibility with enterprise-grade speed and durability built in from the ground up.

Amazon Aurora: High Availability & Performance

What if you need the familiarity of MySQL or PostgreSQL, but your workload demands extreme speed and continuous availability? Aurora delivers both — and it was built natively for the cloud.

5x

MySQL Performance

Aurora delivers up to 5× the throughput of standard MySQL at a fraction of commercial database costs.

3x

PostgreSQL Performance

Achieves up to 3× the throughput of standard PostgreSQL with full compatibility.

6

Data Copies

Aurora replicates 6 copies of your data across 3 Availability Zones for maximum durability.

128TB

Max Storage

Storage automatically grows in 10 GB increments — no manual volume management required.

How Aurora Achieves Its Performance

Decoupled Compute & Storage

Aurora's secret weapon: it **separates the database compute layer from storage**. The database volume is automatically split into small chunks distributed across hundreds of storage nodes, enabling massively parallel I/O operations that traditional architectures can't match.

Even if two full copies become unavailable, Aurora can still process database writes without dropping service — because 4 of 6 copies remain intact.

Aurora vs. Standard MySQL

- Up to 5× faster read/write throughput
- Automatic storage growth — no manual resizing
- 6 copies across 3 AZs vs. 1 copy on 1 server
- Self-healing storage — corrupted data is automatically repaired
- Up to 15 low-latency read replicas
- Fully compatible with MySQL and PostgreSQL drivers

Aurora Analogy

The Synchronized Kitchen Network

Aurora = Synchronized Kitchen Chain

Imagine six prep stations sharing one automated pantry storage inventory. Cooks at every station can pull ingredients simultaneously without waiting on each other. Even if one station loses power, the other five keep cooking without missing a beat. That's Aurora's distributed storage replication — always available, always consistent.

Aurora = High-Tech Auto-Library

Every book automatically duplicates six copies across three library branches simultaneously. Even if one branch experiences a full power outage, patrons at the other branches can still access every title. Researchers never notice the disruption — service continues seamlessly.

Chapter 4

Amazon DynamoDB

Serverless. Millisecond fast. Infinitely scalable. DynamoDB handles key-value workloads at any scale — without a single server to manage.

Amazon DynamoDB: Serverless NoSQL

When scale and raw speed are your top priorities, DynamoDB is the answer. It's fully managed and serverless — you create a table and start reading and writing data. No servers, no clusters, no OS to configure.

Millisecond Latency

DynamoDB guarantees **single-digit millisecond response times** whether you have 10 users or 10 million. Latency doesn't degrade as you scale.

🔑 Key-Value Model

Every item is accessed via a **Partition Key**. Think of it like a coat check — hand in your ticket number, instantly get your coat back. No complex queries needed.

Fully Serverless

No virtual machines, no OS selection, no cluster management. AWS handles all infrastructure — you pay only for what you use.

📊 Flexible Capacity

Choose **On-Demand mode** for unpredictable traffic spikes or **Provisioned Throughput** for steady workloads where cost optimization matters.

DynamoDB Deep Dive

Provisioned Throughput vs. On-Demand Mode

On-Demand Mode

DynamoDB automatically scales capacity up or down to match your actual traffic — with no capacity planning required. You pay per request. Ideal for:

- New applications with unknown traffic patterns
- Flash sale events with massive, unpredictable spikes
- Workloads with many idle hours between bursts

Best for: unpredictable, spiky, or new workloads.

Provisioned Throughput

You specify the exact number of **Read Capacity Units (RCUs)** and **Write Capacity Units (WCUs)** per second. AWS reserves that capacity. Ideal for:

- Applications with consistent, predictable traffic
- Cost optimization on stable workloads
- Auto-scaling policies with defined upper limits

Analogy: Reserving a specific number of delivery locker service doors per hour to handle expected package volume smoothly.

DynamoDB Use Cases

Where DynamoDB Shines

🎮 Gaming Leaderboards

Real-time score lookups for millions of players with sub-10ms response times.

🛒 Shopping Carts

Temporary, high-write session data that doesn't need complex relational joins.

📱 Mobile Backends

User profile lookups, push notification queues, and app state management at scale.

📢 Ad Tech

Millisecond ad targeting decisions across billions of user profiles per day.

DynamoDB Analogy

The Coat Check & The Drive-Thru

Coat Check = Key-Value

You hand the attendant your ticket number — that's your **Partition Key**. They instantly return your coat — that's your **Value**. No cross-referencing your purchase history. Just your key, your result, in milliseconds.

Drive-Thru = NoSQL Speed

The kitchen worker scans your order number and instantly sees what goes in your bag. The order doesn't link to past dining histories or billing profiles. It's built for **maximum speed and throughput** — not complex relational queries.

Chapter 5

Amazon Redshift

When business leaders need answers from years of data across thousands of locations — Redshift delivers petabyte-scale analytics in seconds, not days.

Amazon Redshift: Cloud Data Warehousing

Standard databases like RDS handle fast daily transactions. But when an executive asks, "What were our top-selling products across 500 locations over the last five years?" — that query belongs in a **data warehouse**, not an operational database.

Columnar Storage

Redshift stores data by column rather than by row, enabling lightning-fast aggregation across massive datasets without reading irrelevant fields.

Massively Parallel Processing

Queries are distributed across multiple compute nodes simultaneously, slashing analytical query times from hours to seconds.

Petabyte Scale

Redshift handles petabytes of historical data — years of transactions, logs, and records — designed for analytical workloads, not transactional ones.

BI Integration

Feeds business intelligence tools like Tableau, QuickSight, and Power BI without impacting live customer-facing applications.

Critical Concept

OLTP vs. OLAP — Know the Difference

OLTP — Online Transaction Processing

Services: RDS, Aurora, DynamoDB

Designed for fast, frequent, small reads and writes. Every customer interaction — adding to a cart, updating an address, processing a payment — is an OLTP operation.

- High volume, low complexity per query
- Millisecond response for individual records
- Optimized for concurrent users

OLAP — Online Analytical Processing

Service: Amazon Redshift

Designed for complex, large-scale aggregate queries across historical data. Analysts and executives use OLAP to answer strategic business questions — not individual customer transactions.

- Low volume, extremely high complexity per query
- Scans billions of rows for aggregate results
- Separated from live transactional workloads

Running heavy OLAP queries on an OLTP database degrades performance for live users. Always separate them.

Redshift Analogy

The City Archive vs. The Restaurant HQ

Redshift = City Historical Archive

Instead of checking out individual books, researchers use **parallel scanning machines** to analyze multi-year trend reports across millions of archived documents at once. No one interrupts the daily librarian's check-out desk — the archive operates independently.

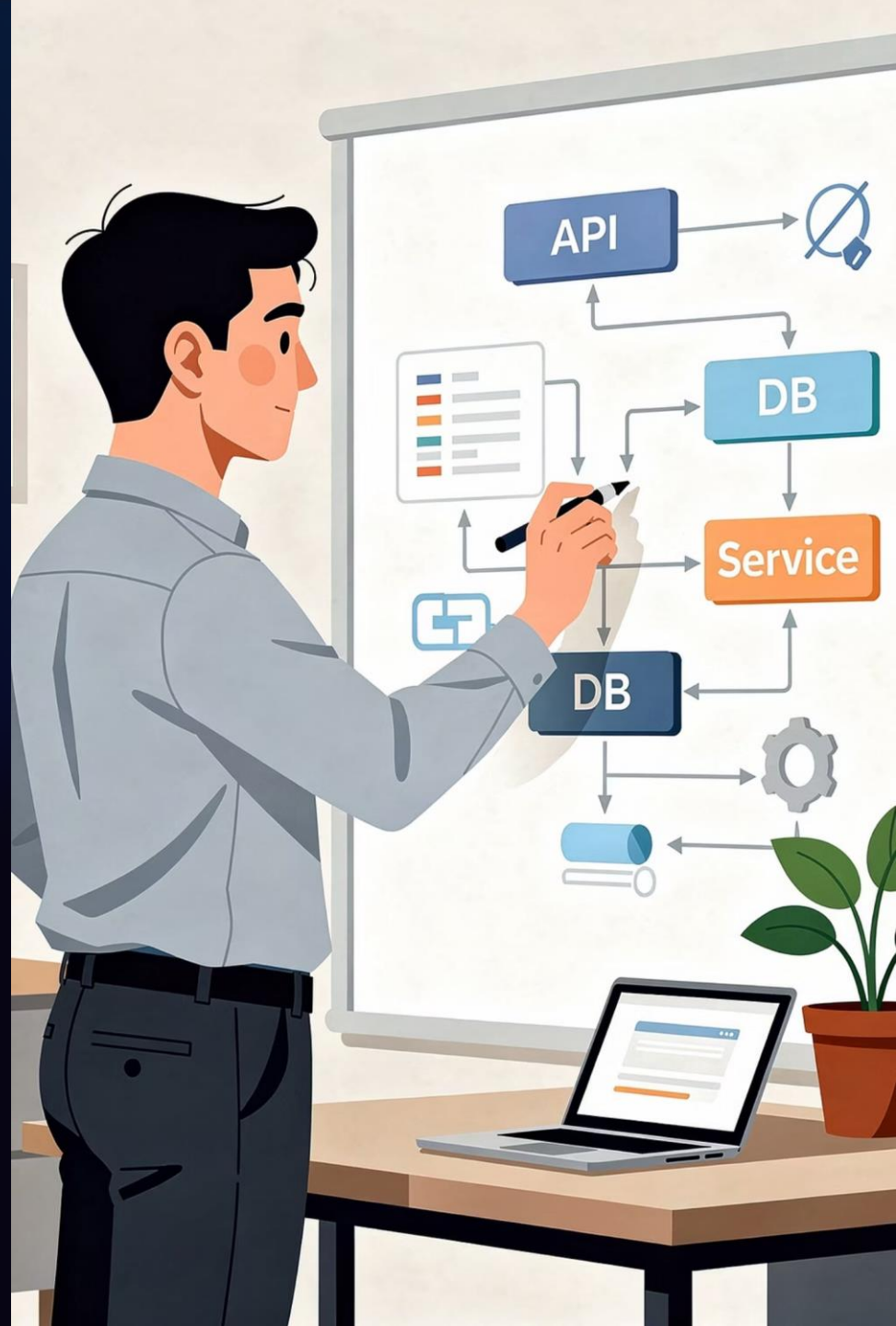
Redshift = Restaurant Corporate HQ

Instead of processing individual meal orders, analysts at headquarters review **quarterly sales reports from 1,000 locations** to determine which menu items to feature next season. This analysis never slows down the kitchen taking customer orders.

Chapter 6

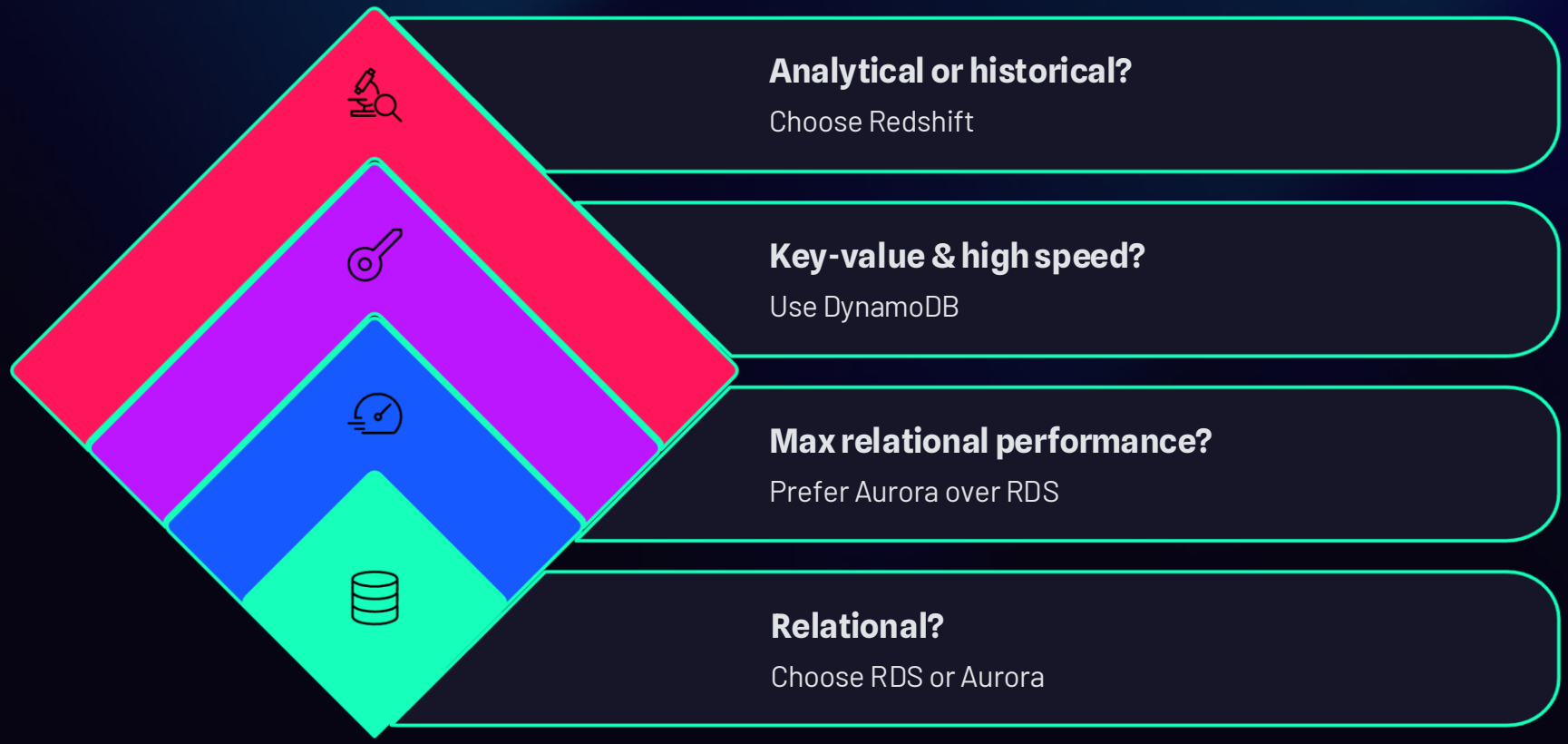
Choosing the Right Tool

One of the most valued skills in cloud architecture is knowing which database to recommend — and why. Let's build that decision-making muscle.



AWS Database Decision Framework

Use these four questions to quickly identify the right AWS database service for any workload scenario — in class, on exams, and on the job.



Remember: AWS's purpose-built philosophy means no single database is best at everything. Matching workload to service is what separates a cloud technician from a cloud architect.

Side-by-Side Service Comparison

Service	Type	Best For	Key Feature
Amazon RDS	Managed Relational	Apps needing SQL with managed ops	Multi-AZ failover, 6 engine options
Amazon Aurora	Cloud-Native Relational	High-performance enterprise SQL workloads	5× MySQL speed, 6 copies across 3 AZs
Amazon DynamoDB	Serverless NoSQL	High-speed key-value at any scale	Single-digit ms latency, serverless
Amazon Redshift	Data Warehouse (OLAP)	Business analytics on historical data	Columnar storage, MPP, petabyte scale

When to Use Each Service

1

Use RDS When...

Your application needs a familiar SQL engine (MySQL, PostgreSQL, SQL Server) with managed infrastructure, automated backups, and Multi-AZ high availability — without the overhead of self-managing servers.

2

Use Aurora When...

You need MySQL or PostgreSQL compatibility but require extreme performance, continuous availability across 3 Availability Zones, and automatic storage scaling up to 128 TB for mission-critical enterprise workloads.

3

Use DynamoDB When...

Your workload demands millisecond response times at massive scale, uses key-value or document access patterns, and benefits from serverless auto-scaling — especially for session data, gaming, or mobile backends.

4

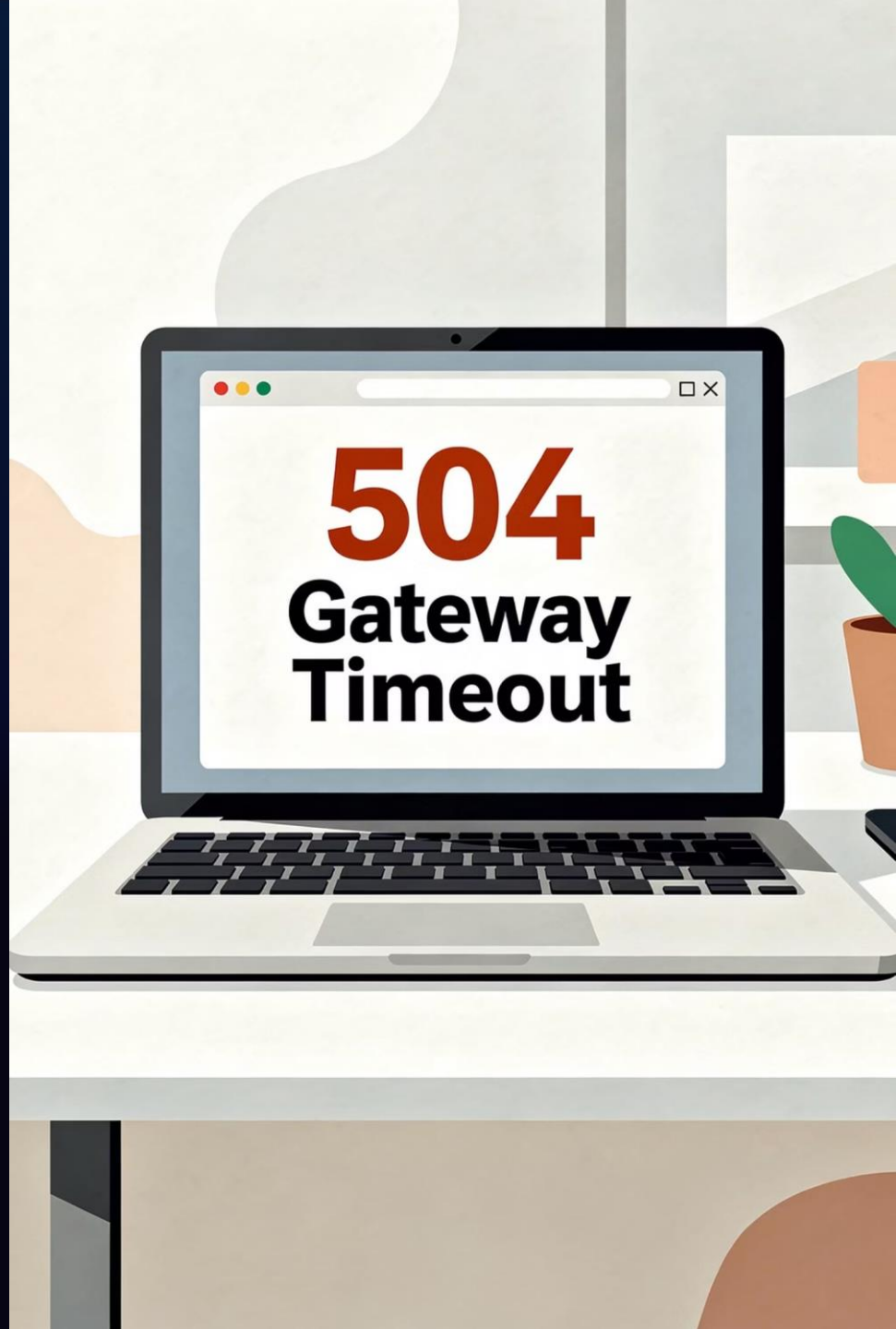
Use Redshift When...

Business analysts need to query petabytes of historical transaction data for strategic reporting, trend analysis, or business intelligence — without impacting the performance of live customer-facing applications.

Chapter 7 — Case Study

The Shopping Cart Crisis

A real-world architectural failure — and the database redesign that could have prevented it. Let's work through this together.



Vocational Case Study

TrendCart: The Black Friday Meltdown

The Setup

TrendCart, a fast-growing e-commerce portal, hosted its entire web platform on a **single Amazon RDS MySQL instance (db.m5.large)**. One database stored everything: customer accounts, product catalog, active shopping carts, and historical order records — all in a single relational schema.

The Trigger

During a promoted Black Friday flash sale, concurrent traffic surged from a typical **2,000 users to over 150,000 concurrent shoppers**. As tens of thousands of shoppers rapidly added items to carts, the database experienced severe row-locking contention.

What Went Wrong

- RDS CPU hit **100% utilization**
- Query latency: 20ms → **12,000ms**
- Connection pools overflowed
- "504 Gateway Timeout" errors worldwide
- Emergency scale-up required a **12-minute reboot**
- Total downtime window: **45 minutes**
- Estimated losses: **$320,000** in sales

Case Study — Root Cause

Why Did RDS Fail Under Load?

Understanding the root cause is key to designing a better architecture. Let's break down exactly why relational databases struggle with high-concurrency shopping cart writes.

1 ACID Compliance = Row-Level Locking

Every time a shopper added or removed an item, RDS executed heavy UPDATE queries with strict row-level locks. With 150,000 concurrent users, thousands of transactions competed for the same locked rows simultaneously — creating a CPU bottleneck.

2 Shopping Carts Are Temporary Data

Shopping cart sessions are **temporary, high-write, low-relationship data**. They don't need foreign key joins to billing history or ACID guarantees. Forcing them into a relational schema adds overhead with no benefit.

3 Scaling RDS Requires Downtime

Vertical scaling (upgrading the instance type) requires a database reboot — a 12-minute outage during peak Black Friday sales. NoSQL services like DynamoDB scale horizontally without any downtime.

Case Study — Debate Question 1

How Would DynamoDB Solve This?

The Problem (RDS)

Relational RDS enforces ACID compliance and row-level locking on every cart write. With 150,000 concurrent users, this creates cascading CPU bottlenecks. The database can't process writes fast enough, so queries queue up and latency explodes from 20ms to 12,000ms.

The Solution (DynamoDB)

Migrating shopping cart session data to DynamoDB eliminates row-locking entirely. Each cart is a **key-value item** accessed by a Partition Key (Customer ID). DynamoDB delivers:

- Single-digit millisecond latency regardless of concurrent users
- On-Demand mode absorbs the Black Friday traffic spike automatically
- No relational lock overhead — writes are instant and independent
- Horizontal scaling with zero downtime

Case Study — Debate Question 2

Separating OLTP from OLAP

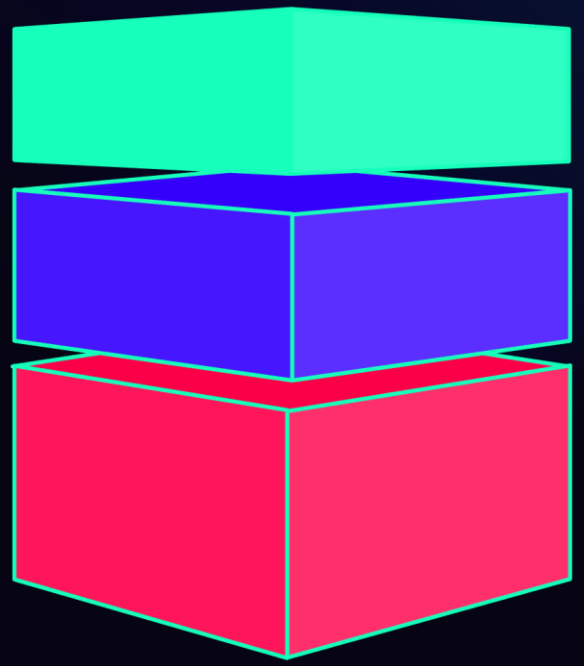
TrendCart's third architectural challenge: running heavy business analytics queries on the same database serving live customer transactions. Here's the recommended three-tier database strategy:

Session Store

DynamoDB for fast shopping cart sessions and low-latency reads.

Analytics Warehouse

Redshift stores historical data for BI and heavy analytics queries.



Transactional Core

Aurora hosts orders, accounts, and billing with ACID guarantees.

This architecture keeps each database doing what it does best — ensuring customer-facing performance is never impacted by executive-level reporting queries.

Case Study — Model Solution

The Recommended Architecture for TrendCart

🔑 DynamoDB

Shopping Cart Sessions

Key-value storage for active cart data. On-Demand mode absorbs any traffic spike with single-digit ms latency. No row locks. No reboot. No downtime.

🗄️ Aurora (or RDS)

Core Relational Data

Customer accounts, order processing, payment billing, and inventory require strict ACID compliance and relational joins. Keep this on a managed relational engine with Multi-AZ enabled.

📊 Redshift

Business Analytics

Export historical order records to Redshift for executive reporting. Heavy analytical queries (top products by region, year-over-year revenue) run here — never on the live operational database.

Grading Checklist

What a Complete Answer Includes

When your instructor grades your case study response, they're looking for these five key elements. Use this checklist to self-assess before submitting.

- ○ Explain that RDS enforces ACID compliance and row-level locking, creating CPU bottlenecks during high-concurrency write spikes on temporary session data like shopping carts.
- ○ Identify Amazon DynamoDB as the ideal NoSQL key-value store for shopping cart data — single-digit millisecond latency, no relational lock overhead, scales horizontally.
- ○ Note that DynamoDB On-Demand mode or auto-scaled Provisioned Throughput absorbs unpredictable flash-sale traffic surges without manual intervention or downtime.
- ○ Recommend maintaining RDS or Aurora for core relational data (orders, billing, customer accounts) while offloading high-volume session writes to DynamoDB.
- ○ Suggest exporting historical order records to Amazon Redshift for OLAP analytics — ensuring heavy analytical queries never degrade live customer transactions.

Career Connection

Why This Matters for Your Cloud Career

Database architecture decisions are among the most consequential choices a cloud technician makes. Employers across every industry — healthcare, retail, finance, logistics — need professionals who can match workloads to the right database tool. This isn't just exam knowledge. These are the conversations happening in real cloud teams right now.

💼 Cloud Support Technician

Diagnose database performance issues, recommend service changes, and implement Multi-AZ configurations for client environments.

Cloud Solutions Architect

Design multi-service database architectures that separate OLTP, NoSQL, and OLAP workloads for enterprise clients.

🛠️ Database Administrator (DBA)

Manage RDS and Aurora instances, configure backup policies, monitor throughput, and optimize query performance across AWS environments.

AWS Certifications — Your Next Step

These Services Appear on AWS Exams

Certification Relevance

Every service covered today — RDS, Aurora, DynamoDB, and Redshift — is a tested topic on the AWS Cloud Practitioner and AWS Solutions Architect Associate exams. Mastering this content in ITNW 1009 gives you a real head start.

Students who earn the AWS Cloud Practitioner certification before graduation report significantly stronger job placement outcomes and higher starting salaries.

What You're Building Toward

- ☑️ **AWS Cloud Practitioner (CLF-C02)** — foundational database concepts
- ☑️ **AWS Solutions Architect Associate (SAA-C03)** — architecture design with database services
- ☑️ **AWS Database Specialty** — advanced database management and optimization

Each course in ITNW 1009 builds toward these credentials. You're already on the path — keep going.

Key Vocabulary Review

Terms From Today's Lesson

Relational

Rigid table schemas with rows, columns, and primary/foreign key relationships enforced by SQL.

NoSQL

Flexible schema databases using key-value pairs or documents — built for scale and millisecond speed.

RDS

AWS managed relational database service supporting MySQL, PostgreSQL, Oracle, SQL Server, and Aurora.

DynamoDB

AWS serverless NoSQL key-value database with single-digit millisecond latency at any scale.

Amazon Aurora

AWS cloud-native relational engine with 5× MySQL performance and 6-copy replication across 3 AZs.

Amazon Redshift

Petabyte-scale columnar data warehouse using MPP for OLAP analytics and business intelligence.

Provisioned Throughput

Pre-reserving a fixed number of DynamoDB read/write capacity units per second for steady workloads.

Data Warehouse

A centralized analytical repository optimized for complex aggregate queries on large historical datasets.

Day 9 — Summary & Takeaways

What You Learned Today

Relational vs. NoSQL

Rigid SQL schemas for accuracy and relationships; flexible NoSQL for speed and scale. Both have essential roles in cloud architecture.

Amazon RDS

Managed relational service with Multi-AZ failover, automated backups, and 6 engine options. Never manually patch a database server again.

Amazon Aurora

Cloud-native relational engine with 5× MySQL performance, 6 copies across 3 AZs, and auto-scaling storage up to 128 TB.

Amazon DynamoDB

Serverless NoSQL for key-value workloads. Single-digit millisecond latency at any scale, with On-Demand or Provisioned capacity modes.

Amazon Redshift

Petabyte-scale OLAP data warehouse using columnar storage and MPP. Separate your analytics from live transactional workloads.

Real-World Application

Purpose-built databases prevent costly outages like the TrendCart Black Friday failure. Choosing the right service is a critical, employable skill.

You're Making Real Progress

Keep Going — You've Got This.

Databases are at the heart of every cloud-powered application in the world. You now understand how AWS's purpose-built approach separates the right tool for the right job — and that knowledge makes you more valuable to every employer in the industry. See you in Day 10!

☆ ITNW 1009 — CLOUD ESSENTIALS

DAY 9 COMPLETE